Blueprint

Security & Architecture Brief · an auditable, self-hosted UI layer for regulated teams

THE ARCHITECTURE, IN ONE LINE

No eval → the contract describes the **screen, not the behavior** → you self-host it. Everything below follows from that.

What it is

An open UI **contract** (JSON) plus a thin **runtime** you host yourself. The runtime walks each node and maps it to **your own React components** — it is not an interpreter, and it generates no code from strings. Real logic stays in typed, reviewable code; the JSON only describes structure.

Why it stays out of your audit scope

- **No eval / no code-from-strings.** The runtime validates a contract with zero dynamic code — it runs under a strict Content-Security-Policy with no `unsafe-eval`. Nothing is generated from strings, so what passes review is what ships.
- **Self-hosted, zero external calls.** It runs inside your boundary — no data egress, no vendor endpoint, nothing to allow-list outbound. It drops into an isolated subnet or an air-gap unchanged.
- **Versioned, diffable contract.** Every change to a regulated screen is a reviewable pull-request diff — “prove this screen behaves” becomes a diff you hand an auditor, not a promise.
- **Open source (AGPL) + commercial license.** Your team can read every line before it ships. A commercial (non-AGPL) license is available for closed-source products.

The single most attackable claim here is “no eval.” So we prove it, rather than assert it — see *Verify it yourself* overleaf.

WHERE IT LANDS IN COMPLIANCE

These are **deployment properties that enable mandates** — not certifications Blueprint holds. A library you run isn't a SaaS in your scope; the authorization is your system's.

PCI DSS 6.4.3 / 11.6.1	Every script on a payment page inventoried, justified, tamper-monitored. Blueprint generates nothing from strings — the inventory is short and the CSP stays strict.
Data residency / sovereignty	You run it on your own infrastructure, in your own region. There is no vendor endpoint in another country to account for.
NIST SC-7 / AC-4 · isolation	Control and isolate flows at the boundary. The runtime makes zero external calls, so there is nothing to egress from an isolated or classified enclave.

VERIFY IT YOURSELF — FROM SOURCE, NOT A HOSTED PAGE

There is no demo server to trust. The runtime is open source — clone it and run the proof on your own machine, against your own contract.

- **Live no-eval proof.** Open `packages/blueprint-runtime/security/csp-no-eval.html`: the real validator runs under a strict CSP (no `unsafe-eval`) — zero violations, and a deliberate `new Function()` blocked by the browser. Drop in your own contract and prove it on your screens.
- **Zero-eval CI test.** `pnpm --filter @dashforge/blueprint-core test` swaps `Function`/`eval` for throwing stubs and asserts they are never called — it fails loudly on any regression.
- **Dependency SBOM + full source.** The production closure is small and auditable (`SBOM.md`). Read every line: github.com/kensaadi/blueprint — AGPL-3.0.

WHAT WE CLAIM — AND WHAT WE DON'T

- **No eval** in the payload — nothing generated from strings.
- **No data egress** — self-hosted, runs in your environment.
- The contract is **versioned, diffable, CSP-clean**.

× **Not SOC 2 / PCI / ISO certified** — a library you run isn't a SaaS in your scope.

× We **don't process or store your data** — nothing of yours for us to lose.

× We **don't make you compliant** — we make your compliance *provable*.

Talk to us about your audit — **info@dashforge-ui.com** · As of August 2026. Blueprint is pre-1.0; the claims above are architectural and verifiable today.